

# DCDeepTech AI Gateway

Alibaba Cloud Singapore × SophNet China QwenVL  
阿里云新加坡节点 跨境接入中国 QwenVL 实验指导书

## Lab Manual 实验手册

Version 1.0 | Bilingual EN / ZH 中英对照

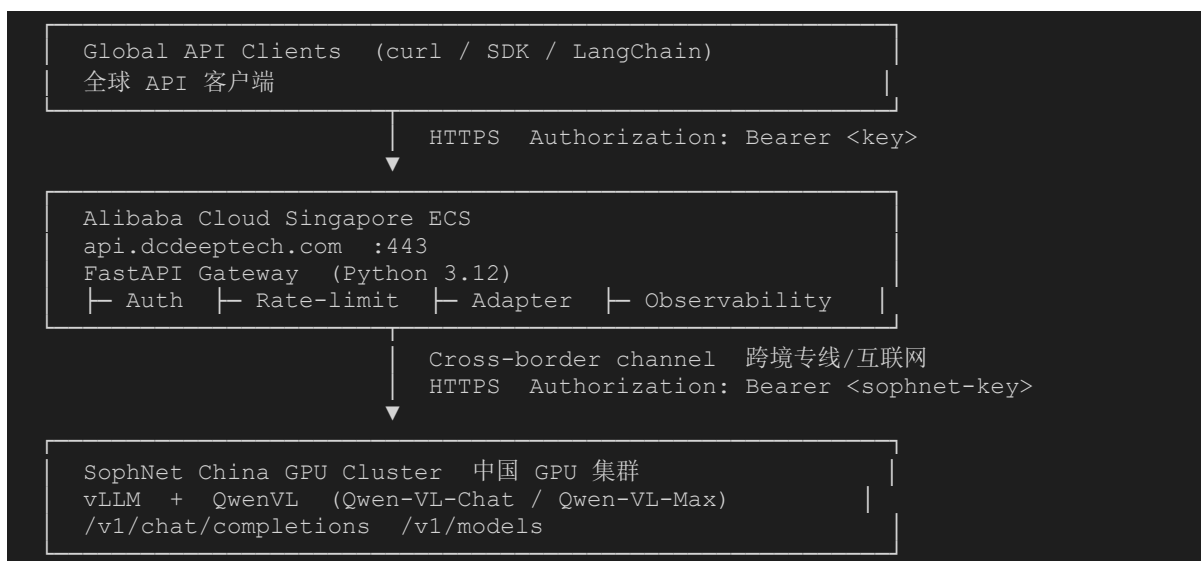
|                        |                                   |
|------------------------|-----------------------------------|
| Gateway Domain 网关域名    | api.dcddeeptech.com               |
| Deployment Region 部署区域 | Singapore (ap-southeast-1)<br>新加坡 |
| Upstream Model 上游模型    | SophNet QwenVL (China)            |
| API Compatibility 接口兼容 | OpenAI Chat Completions v1        |
| Document Date 文档日期     | 2026-03-21                        |

## 1. Overview 概述

This lab guide walks you through deploying the DCDeepTech AI Gateway on Alibaba Cloud Singapore (ECS), configuring it to connect to SophNet's China-side QwenVL inference backend, and validating the end-to-end path with text and multimodal (image) requests. 本实验手册指导您在阿里云新加坡节点（ECS）上部署 DCDeepTech AI Gateway，配置其连接至 SophNet 中国侧 QwenVL 推理后端，并通过文本与多模态（图片）请求验证全链路连通性。

### 1.1 Architecture 架构图

The gateway sits in Singapore and acts as an OpenAI-compatible reverse proxy toward the China GPU cluster. 网关部署于新加坡，作为面向中国 GPU 集群的 OpenAI 兼容反向代理层。



## 2. Pre-requisites 前置条件

### 2.1 Accounts & Access 账号与访问

- Alibaba Cloud account with ECS access in ap-southeast-1 (Singapore) 拥有阿里云账号，可在 ap-southeast-1（新加坡）创建 ECS
- SophNet API credentials: base URL + API key SophNet API 凭证：接入地址 + API Key
- A registered domain or the ECS public IP for api.dcdeeptech.com 已有域名或使用 ECS 公网 IP 作为接入地址
- SSH key pair imported into Alibaba Cloud 已在阿里云导入 SSH 密钥对

### 2.2 Local Tools 本地工具

- curl ≥ 7.68 or Python ≥ 3.9 with openai SDK curl ≥ 7.68 或安装了 openai SDK 的 Python ≥ 3.9
- SSH client (Terminal / PuTTY) SSH 客户端 (Terminal / PuTTY)
- Optional: Docker Desktop for local pre-test 可选：Docker Desktop 用于本地预验证



All commands in this guide are run on the ECS instance unless stated otherwise.  
除非特别说明，本指导书中的所有命令均在 ECS 实例上执行。

## 3. Alibaba Cloud ECS Setup 阿里云 ECS 配置

### 3.1 Create the ECS Instance 创建 ECS 实例

1. Log in to the Alibaba Cloud Console → ECS → Instances → Create Instance. 登录阿里云控制台 → ECS → 实例 → 创建实例。
2. Select Region: Singapore (ap-southeast-1). 选择地域：新加坡（ap-southeast-1）。
3. Choose instance type: ecs.c7.xlarge (4 vCPU, 8 GiB) or above. 选择实例规格：ecs.c7.xlarge（4 vCPU，8 GiB）或更高。
4. Select OS: Ubuntu 22.04 LTS 64-bit. 操作系统：Ubuntu 22.04 LTS 64-bit。
5. Assign a public IP (or Elastic IP). Note the IP address. 分配公网 IP（或弹性公网 IP），记录 IP 地址。
6. Open Security Group ports: 22 (SSH), 80 (HTTP), 443 (HTTPS), 8000 (gateway dev). 安全组开放端口：22（SSH）、80（HTTP）、443（HTTPS）、8000（网关调试）。



For production use port 443 with a TLS certificate. Port 8000 is for lab validation only.  
生产环境请使用 443 端口并配置 TLS 证书，8000 端口仅用于实验验证。

## 3.2 Connect via SSH SSH 连接

```
# Replace <ECS_IP> with your instance public IP
# 将 <ECS_IP> 替换为您的实例公网 IP

ssh -i ~/.ssh/your_key.pem ubuntu@<ECS_IP>
```

## 3.3 Install System Dependencies 安装系统依赖

```
# Update package index 更新包索引
sudo apt-get update && sudo apt-get upgrade -y

# Install Python 3.12, pip, git, curl 安装 Python 3.12、pip、git、curl
sudo apt-get install -y python3.12 python3.12-venv python3-pip git curl

# Confirm Python version 确认 Python 版本
python3.12 --version
# Expected: Python 3.12.x 期望输出: Python 3.12.x
```

# 4. Deploy the Gateway 部署网关

## 4.1 Clone the Repository 克隆代码仓库

```
cd ~
git clone https://github.com/dcdeeptech/ai-gateway.git
cd ai-gateway
```

If you received the gateway as a zip archive instead: 如果是通过 zip 包交付：

```
scp -i ~/.ssh/your_key.pem gateway.zip ubuntu@<ECS_IP>:~
unzip gateway.zip && cd dcdeeptech-gateway
```

## 4.2 Create Virtual Environment 创建虚拟环境

```
python3.12 -m venv .venv
source .venv/bin/activate

# Install dependencies 安装依赖
pip install --upgrade pip
pip install -r requirements.txt
```

## 4.3 Configure Environment Variables 配置环境变量

Copy the example file and fill in your real values: 复制示例文件并填入真实值:

```
cp .env.example .env
nano .env # or: vim .env
```

Edit the following variables: 编辑以下变量:

| Variable 变量         | Example Value 示例值        | Description 说明                                     |
|---------------------|--------------------------|----------------------------------------------------|
| GATEWAY_API_KEY     | sg-gw-xxxxxxxxxxxxx      | Secret token your clients will present 客户端鉴权 Token |
| GATEWAY_HOST        | 0.0.0.0                  | Bind address (keep as-is) 绑定地址（保持默认）               |
| GATEWAY_PORT        | 8000                     | Listen port 监听端口                                   |
| SOPHNET_API_URL     | https://api.sophnet.com  | SophNet base URL SophNet 接入地址                      |
| SOPHNET_API_KEY     | sk-sophnet-xxxxxxxxxxxxx | SophNet API key SophNet API Key                    |
| SOPHNET_AUTH_MODE   | bearer                   | Auth mode for upstream 上游鉴权方式                      |
| DEFAULT_MODEL       | qwenvl                   | Fallback model name 默认模型名称                         |
| REQUEST_TIMEOUT     | 60                       | Upstream timeout seconds 上游超时（秒）                   |
| RATE_LIMIT_ENABLED  | true                     | Enable rate limiting 启用速率限制                        |
| RATE_LIMIT_REQUESTS | 60                       | Requests per window 每窗口请求数                         |
| RATE_LIMIT_WINDOW   | 60                       | Window size in seconds 窗口大小（秒）                     |



Never commit your .env file to git. The .gitignore already excludes it.

请勿将 .env 文件提交至 git 仓库，.gitignore 已默认排除该文件。

## 4.4 Start the Gateway 启动网关

For a quick lab test (foreground): 实验快速测试（前台运行）：

```
source .venv/bin/activate
python main.py

# Expected output 期望输出:
# INFO      [main] Gateway starting - upstream: https://api.sophnet.com
# INFO      default_model: qwenvl
# INFO      Uvicorn running on http://0.0.0.0:8000 (Press CTRL+C to quit)
```

For persistent background operation (using systemd): 持久后台运行（使用 systemd）：

```
# Create service file 创建服务文件
sudo tee /etc/systemd/system/ai-gateway.service > /dev/null <<EOF
[Unit]
Description=DCDeepTech AI Gateway
After=network.target

[Service]
User=ubuntu
WorkingDirectory=/home/ubuntu/ai-gateway
EnvironmentFile=/home/ubuntu/ai-gateway/.env
ExecStart=/home/ubuntu/ai-gateway/.venv/bin/uvicorn main:app --host 0.0.0.0 --port 8000
Restart=always
RestartSec=5

[Install]
WantedBy=multi-user.target
EOF

sudo systemctl daemon-reload
sudo systemctl enable ai-gateway
sudo systemctl start ai-gateway
sudo systemctl status ai-gateway
```

## 5. Validation Tests 验证测试

Run each test in sequence. A ✓ next to each step confirms the system is working correctly. 按顺序执行以下测试，每步旁边的 ✓ 表示系统工作正常。

Set these shell variables once before running all tests: 在运行所有测试前，先设置以下 Shell 变量：

```
export GW_HOST="http://<ECS_IP>:8000" # or https://api.dcddeeptech.com
export GW_KEY="sg-gw-xxxxxxxxxxxxx" # your GATEWAY API KEY value
```

### Step 1 Health Check (no auth required) 健康检查（无需鉴权）

Command 命令:

```
curl -s $GW_HOST/health | python3 -m json.tool
```

Expected response 期望响应:

```
{
  "status": "ok",
  "service": "DCDeepTech AI Gateway",
  "region": "Singapore",
  "timestamp": 1710000000,
  "upstream": {
    "url": "https://api.sophnet.com",
    "status": 200,
    "reachable": true
  }
}
```



Pass criteria: HTTP 200, status = "ok", upstream.reachable = true.  
通过条件: HTTP 200, status 为 "ok", upstream.reachable 为 true。

## Step 2 Authentication Rejection Test 鉴权拒绝测试

Verify that missing or wrong tokens are rejected: 验证缺失或错误 Token 被拒绝:

```
# No token - expect 401 无 Token, 期望 401
curl -s -o /dev/null -w '%{http_code}' $GW_HOST/v1/models
# Output: 401

# Wrong token - expect 401 错误 Token, 期望 401
curl -s -o /dev/null -w '%{http_code}' $GW_HOST/v1/models \
-H 'Authorization: Bearer wrong-key'
# Output: 401
```



Pass criteria: both requests return HTTP 401.  
通过条件: 两个请求均返回 HTTP 401。

## Step 3 Model Listing 模型列表

```
curl -s $GW_HOST/v1/models \
-H "Authorization: Bearer $GW_KEY" | python3 -m json.tool
```

Expected response 期望响应:

```
{
  "object": "list",
  "data": [
    {
      "id": "qwenvl",
      "object": "model",
    }
  ]
}
```

```
      "created": 1710000000,
      "owned_by": "dcdeeptech"
    }
  ]
}
```



Pass criteria: HTTP 200, data array contains at least one model with id = qwenvl.  
通过条件: HTTP 200, data 数组包含至少一条 id 为 qwenvl 的模型记录。

## Step 4 Text Chat Completion 文本对话

```
curl -s $GW_HOST/v1/chat/completions \
-H "Authorization: Bearer $GW_KEY" \
-H "Content-Type: application/json" \
-d '{
  "model": "qwenvl",
  "messages": [
    {
      "role": "user",
      "content": "What is the capital of Singapore? Reply in one sentence."
    }
  ],
  "max_tokens": 128
}' | python3 -m json.tool
```

Expected response (abbreviated) 期望响应 (节略):

```
{
  "id": "chatcmpl-...",
  "object": "chat.completion",
  "model": "qwenvl",
  "choices": [
    {
      "index": 0,
      "message": {
        "role": "assistant",
        "content": "The capital of Singapore is Singapore City."
      },
      "finish_reason": "stop"
    }
  ],
  "usage": { "prompt_tokens": 18, "completion_tokens": 10, "total_tokens": 28 }
}
```



Pass criteria: HTTP 200, choices[0].message.content is a non-empty string, finish\_reason = stop.  
通过条件: HTTP 200, choices[0].message.content 为非空字符串, finish\_reason 为 stop。

## Step 5 Streaming Chat Completion 流式对话

Test Server-Sent Events (SSE) streaming — the response should arrive as a token stream. 测试 SSE 流式输出, 响应应以 token 流形式逐步到达。

```
curl -s $GW_HOST/v1/chat/completions \
-H "Authorization: Bearer $GW_KEY" \
-H "Content-Type: application/json" \
-d '{
  "model": "qwenvl",
  "stream": true,
  "messages": [{"role": "user", "content": "Count from 1 to 5 slowly."}],
  "max_tokens": 64
}'
```

Expected output format 期望输出格式:

```
data: {"id":"chatcmpl-...","choices":[{"delta":{"role":"assistant"},"index":0}]}
data: {"id":"chatcmpl-...","choices":[{"delta":{"content":"1"},"index":0}]}
data: {"id":"chatcmpl-...","choices":[{"delta":{"content":","},"index":0}]}
...
data: [DONE]
```

✓ Pass criteria: Content-Type header is text/event-stream, multiple data: lines arrive, ends with data: [DONE].  
通过条件: Content-Type 为 text/event-stream, 收到多行 data: 事件, 以 data: [DONE] 结尾。

## Step 6 Multimodal — QwenVL Image Input 多模态 — QwenVL 图片输入

This test verifies the full Qwen-VL multimodal path: image\_url content part is forwarded correctly to the China backend. 此测试验证完整的 Qwen-VL 多模态链路: image\_url 内容部分被正确转发至中国后端。

Using a public test image (Singapore skyline): 使用公开测试图片（新加坡天际线）:

```
curl -s $GW_HOST/v1/chat/completions \
-H "Authorization: Bearer $GW_KEY" \
-H "Content-Type: application/json" \
-d '{
  "model": "qwenvl",
  "messages": [
    {
      "role": "user",
      "content": [
        {
          "type": "image_url",
          "image_url": {
            "url":
"https://upload.wikimedia.org/wikipedia/commons/thumb/1/1c/Singapore_Skyline_2019.jpg/1280px-Singapore_Skyline_2019.jpg"
          }
        },
        {
          "type": "text",
          "text": "Describe what you see in this image in 2 sentences."
        }
      ]
    }
  ],
  "max_tokens": 200
}'
```



```
} ' | python3 -m json.tool
```

Expected: the model returns a description of the image. 期望：模型返回对图片内容的描述。

✓ Pass criteria: HTTP 200, choices[0].message.content describes the image content (not an error message).  
通过条件：HTTP 200, choices[0].message.content 描述了图片内容（非错误信息）。

## Step 7 OpenAI Python SDK Compatibility OpenAI Python SDK 兼容性验证

Verify that the gateway works as a drop-in replacement for the OpenAI API using the official Python SDK. 验证网关可作为 OpenAI API 的无缝替换，使用官方 Python SDK 接入。

```
# Install openai SDK on ECS (or run locally) 在 ECS 或本地安装 openai SDK
pip install openai
```

Create a test script: 创建测试脚本: undefined

```
cat > test_sdk.py << 'EOF'
import os
from openai import OpenAI

client = OpenAI(
    base_url=os.getenv("GW_HOST") + "/v1",
    api_key=os.getenv("GW_KEY"),
)

# — Text chat 文本对话 —————
print('=== Text Chat 文本对话 ===')
resp = client.chat.completions.create(
    model="qwenvl",
    messages=[{"role": "user", "content": "Hello from Singapore!"}],
    max_tokens=64,
)
print(resp.choices[0].message.content)
print(f'Tokens used: {resp.usage.total_tokens}')

# — Streaming 流式输出 —————
print('\n=== Streaming 流式输出 ===')
stream = client.chat.completions.create(
    model="qwenvl",
    messages=[{"role": "user", "content": "Say hello in 3 languages."}],
    max_tokens=128,
    stream=True,
)
for chunk in stream:
    delta = chunk.choices[0].delta.content or ''
    print(delta, end='', flush=True)
print()

# — Multimodal 多模态 —————
print('\n=== Multimodal 多模态 ===')
resp2 = client.chat.completions.create(
```

```

model="qwenvl",
messages=[{
    "role": "user",
    "content": [
        {"type": "image_url", "image_url": {"url":
"https://upload.wikimedia.org/wikipedia/commons/thumb/1/1c/Singapore_Skyline_2019.jpg/1280px-Singapore_Skyline_2019.jpg"}},
        {"type": "text", "text": "What city is shown? Reply in one word."}
    ]
}],
max_tokens=32,
)
print(resp2.choices[0].message.content)
EOF

python3 test_sdk.py

```



Pass criteria: all three sections print non-empty model responses without exceptions.  
通过条件：三个部分均打印非空的模型响应，无异常抛出。

## 6. Troubleshooting 常见问题排查

### 6.1 Connection Refused on Port 8000 8000 端口连接被拒

- Check Security Group: port 8000 must be open for your source IP. 检查安全组：需为您的来源 IP 开放 8000 端口。
- Confirm gateway is running: `sudo systemctl status ai-gateway` 确认网关正在运行：`sudo systemctl status ai-gateway`
- Check logs: `sudo journalctl -u ai-gateway -f` 查看日志：`sudo journalctl -u ai-gateway -f`

### 6.2 upstream.reachable = false in /health 健康检查显示上游不可达

- Verify SOPHNET\_API\_URL is correct and reachable from Singapore ECS. 确认 SOPHNET\_API\_URL 正确且可从新加坡 ECS 访问。
- Test directly: `curl -v https://api.sophnet.com/health` 直接测试：`curl -v https://api.sophnet.com/health`
- Check if a cross-border VPN or dedicated line is required. 检查是否需要跨境 VPN 或专线。
- Verify SOPHNET\_API\_KEY is correct and not expired. 确认 SOPHNET\_API\_KEY 正确且未过期。

### 6.3 HTTP 401 from Gateway 网关返回 401

- Confirm GATEWAY\_API\_KEY in .env matches the key in your curl -H or SDK api\_key. 确认 .env 中的 GATEWAY\_API\_KEY 与 curl 请求头或 SDK api\_key 值一致。
- Authorization header must be exactly: "Bearer <key>" (capital B, one space). Authorization 请求头格式必须为："Bearer <key>"（B 大写，一个空格）。

### 6.4 HTTP 504 Gateway Timeout 504 网关超时

- Increase REQUEST\_TIMEOUT in .env (e.g. 120 for vision tasks). 增加 .env 中的 REQUEST\_TIMEOUT（如视觉任务可设为 120）。
- Vision / multimodal inference takes longer — 30s default is often too short. 视觉/多模态推理耗时较长，30 秒默认值通常不足。
- Check upstream response time directly from ECS. 直接从 ECS 测试上游响应时间。

### 6.5 Multimodal Returns Error / Empty Content 多模态返回错误或内容为空

- Confirm the model name exactly matches what SophNet expects (e.g. Qwen-VL-Chat). 确认模型名称与 SophNet 期望的完全一致（如 Qwen-VL-Chat）。
- Verify the image URL is publicly accessible — test with curl -I <url>. 确认图片 URL 可公开访问，使用 curl -I <url> 验证。
- Check SophNet documentation for supported image formats and size limits. 查看 SophNet 文档了解支持的图片格式和大小限制。

## 7. Lab Completion Checklist 实验完成检查清单

Mark each item once confirmed. All items must pass before declaring the lab complete. 完成后逐项打勾确认，所有项目通过后实验方为完成。

| #                        | Test Item 测试项目               | Pass Criteria 通过条件                         |
|--------------------------|------------------------------|--------------------------------------------|
| <input type="checkbox"/> | Step 1 — Health Check 健康检查   | HTTP 200, status="ok", upstream reachable  |
| <input type="checkbox"/> | Step 2 — Auth Rejection 鉴权拒绝 | Both no-token and wrong-token return 401   |
| <input type="checkbox"/> | Step 3 — Model List 模型列表     | HTTP 200, qwenvl appears in data[]         |
| <input type="checkbox"/> | Step 4 — Text Chat 文本对话      | HTTP 200, non-empty assistant content      |
| <input type="checkbox"/> | Step 5 — Streaming 流式输出      | SSE stream arrives, ends with [DONE]       |
| <input type="checkbox"/> | Step 6 — Multimodal 多模态图片    | HTTP 200, image is described correctly     |
| <input type="checkbox"/> | Step 7 — Python SDK SDK 兼容   | All three SDK calls succeed without errors |

|   |                                                                                                                                                      |
|---|------------------------------------------------------------------------------------------------------------------------------------------------------|
| ✓ | When all 7 boxes are checked, the Singapore → China QwenVL pathway is verified and production-ready.<br>当全部 7 项打勾确认后，新加坡至中国 QwenVL 跨境链路验证完成，可投入生产使用。 |
|---|------------------------------------------------------------------------------------------------------------------------------------------------------|